

Repositorio de GitHub:

https://github.com/kevinanton01/Grupo_4_Parcial2_BD2_Div133

PARTE 2

1. Entidades del sistema y relaciones

El sistema de gestión de inventario está compuesto por cuatro colecciones principales que interactúan para mantener la trazabilidad de los productos:

- **Categorías:** Entidad maestra que clasifica los productos.
- **Usuarios:** Entidad que almacena los perfiles de los empleados y administradores.
- **productos:** Entidad central que contiene el stock actual, precios y detalles técnicos.
- **movimientos:** Entidad de transacciones que registra las entradas y salidas de stock, vinculando productos y usuarios.

Relaciones:

- Los **productos** hacen referencia a una **categoría** mediante un `categoriald`.
- Los **movimientos** hacen referencia a un **producto** mediante un `productold`.
- Los **movimientos** integran (embebido) el estado del **usuario** al momento de la operación.

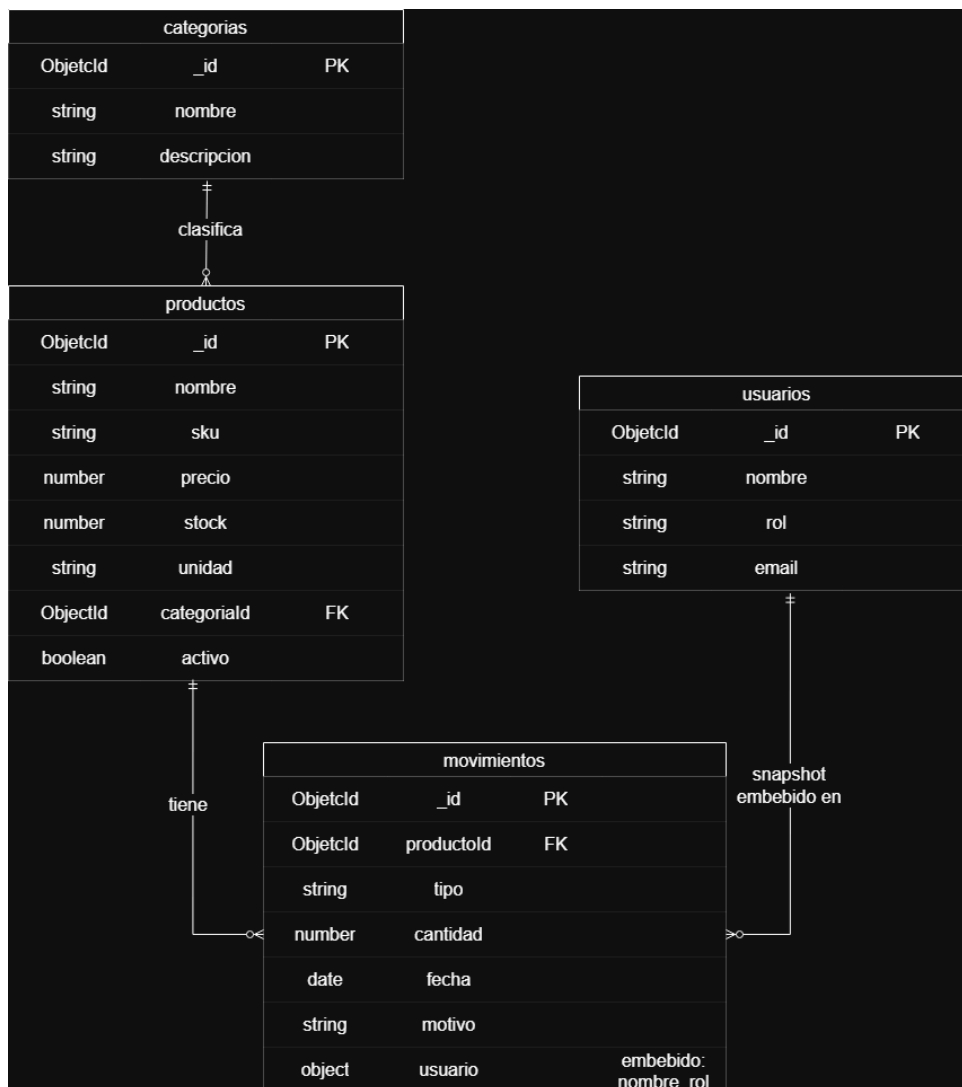
2. Uso de Embedding y Referencing

- **Referencing (Referenciado):** Se utilizó entre productos y categorías, así como entre movimientos y productos.
 - **¿Por qué?** Aplicamos este patrón para normalizar la información y evitar la redundancia. Si el nombre de una categoría cambia, solo debemos actualizar un documento, garantizando la consistencia en todo el catálogo de productos. Asimismo, permite que la colección productos crezca sin incluir datos excesivos.
- **Embedding (Embebido):** Se utilizó para incluir los datos del usuario dentro de la colección movimientos.
 - **¿Por qué?** En este caso, el usuario se guarda como un *snapshot* (instantánea) en el momento del movimiento. Esto es crucial para la auditoría: si un usuario cambia su nombre o rol en la colección usuarios, el histórico de movimientos debe mantener intacta la identidad del responsable tal como estaba al momento de realizar la acción.

3. Análisis comparativo: Modelo Relacional vs. NoSQL

- **Ganancia con NoSQL:** Hemos ganado **agilidad y flexibilidad**. Al usar documentos JSON, podemos añadir campos (como números de lote o proveedores) a los productos o movimientos sin necesidad de ejecutar procesos costosos de *ALTER TABLE* o migraciones complejas típicas de SQL.
- **Pérdida frente al modelo relacional:** Se pierde la integridad referencial automática (tipo *Foreign Key*). En SQL, la base de datos impediría borrar una categoría si esta tiene productos asociados. En este modelo NoSQL, debemos delegar esa validación a la lógica de la aplicación ([Node.js](#)).

Diagrama Entidad relación



PARTE 3

Pipeline de agregación

Para usar la tubería de agregación o pipeline de agregación debemos usar el método `aggregate` sobre la colección en la que queramos. Este método recibe un array de etapas: en la primera etapa entran documentos de la colección y lo que devuelva esa etapa como resultado van a ser documentos que se van a ingresar en la siguiente etapa, y así sucesivamente hasta la última etapa que va a devolver en pantalla el resultado de la operación realizada.

Etapas

- **\$match**: sirve para filtrar documentos de esta etapa de acuerdo a las condiciones que pongamos.
- **\$project**: permite controlar los campos de los documentos de esta etapa que se van a mostrar por pantalla (se les da el valor 1 a los campos que se van a mostrar y se pone el valor 0 en el `_id` si no quieres que se muestre por defecto). También permite modificar el nombre de los campos y usar `concat` para concatenar strings.
- **\$sort**: ordenará los documentos de esta etapa usando los nombres de los campos y un valor (1 para ascendente y -1 para descendente).
- **\$limit**: limita el número de documentos de esta etapa basándose en el valor usado.
- **\$skip**: salta los primeros n documentos de esta etapa.
- **\$group**: permite agrupar documentos por los valores de un campo. El uso del campo `_id` es obligatorio. Dentro se pueden usar operadores como `$sum`, `$avg`, `$min`, `$max` y `$push`.
- **\$unwind**: descompone un campo array en múltiples documentos individuales.
- **\$count**: cuenta cuántos documentos hay en esta etapa y devuelve el resultado en un nuevo campo.
- **\$sortByCount**: agrupa, cuenta y ordena de forma descendente en una sola operación.
- **\$lookup**: permite hacer un join entre dos colecciones. Utiliza:
 - **from**: colección destino.
 - **localField**: campo de la colección actual.
 - **foreignField**: campo de la colección destino.
 - **as**: nombre del nuevo array de salida.

Operadores

- **\$sum**: contar o sumar.
- **\$avg**: calcular el promedio.
- **\$min**: obtener el valor mínimo.
- **\$max**: obtener el valor máximo.
- **\$push**: guardar valores en un array.

Ejemplos

Match

JavaScript

```
db.coleccion.aggregate([ { $match: { edad: { $gt: 18 } } } ] )
```

Filtramos los elementos de la colección por el campo edad, mostrando solo documentos con valor mayor a 18.

Project

JavaScript

```
db.coleccion.aggregate([ { $project: { _id: 0, nombre: 1, apellido: 1 } } ] )
```

Muestra únicamente los campos nombre y apellido, omitiendo el campo _id.

Sort

JavaScript

```
db.coleccion.aggregate([ { $sort: { nombre: 1 } } ] )
```

Muestra los documentos ordenados de forma ascendente por el campo nombre.

Limit

JavaScript

```
db.coleccion.aggregate([ { $limit: 5 } ] )
```

Limita la salida a los primeros 5 documentos de la colección.

Skip

JavaScript

```
db.coleccion.aggregate([ { $skip: 3 } ] )
```

Saltea los primeros 3 documentos y muestra los posteriores.

Group

JavaScript

```
db.coleccion.aggregate([ { $group: { _id: "$programa", total: { $sum: 1 } } } ] )
```

Agrupar los documentos por el campo programa y muestra la cantidad total por grupo.

Lookup

JavaScript

```
db.productos.aggregate([ { $lookup: { from: 'categorias',
localField: 'categoriaId', foreignField: '_id', as: 'categoria' }
}, { $unwind: '$categoria' }, { $project: { _id: 0, nombre: 1,
'categoria.nombre': 1 } } ])
```

traer la categoría de cada producto

PARTE 4

Cheatsheet de JavaScript / Node.js para Mongo — borrador

1. async/await — operaciones que toman tiempo

- `await` pausa la ejecución hasta que la operación asíncrona (conectarse a la base, insertar, consultar) termine.
- No bloquea el resto del programa como pasaría con un loop manual.

JavaScript

```
// connection.js - conectarse es asíncrono, por eso getDb() es
async
async function getDb() {
  if (db) return db;
  client = new MongoClient(URI);
  await client.connect(); // <- espera a que la conexión esté
  lista
  db = client.db(DB_NAME);
  return db;
}
```

- Toda función que usa `await` adentro tiene que estar marcada `async`.
- Si la olvidás, `await` tira error de sintaxis.

2. try/catch con async/await

- Es la forma moderna de manejar errores en operaciones asíncronas.
- Reemplaza al patrón viejo de callbacks con `.then().catch()`.

JavaScript

```
// 00-seed-all.js
seedAll()
  .catch((err) => console.error('Error durante el seed:', err))
  .finally(() => closeConnection());
```

- Patrón alternativo con `try/catch` explícito (útil si necesitás lógica distinta según el tipo de error):

JavaScript

```
async function seedAll() {
  try {
    const db = await getDb();
    await seedCategorias(db);
  } catch (err) {
    console.error('Falló el seed:', err.message);
  } finally {
    await closeConnection(); // se ejecuta SIEMPRE, haya error o no
  }
}
```

3. Promise.all — consultas en paralelo

- Si tenés varias operaciones independientes, `await` una por una es lento porque espera en serie.
- `Promise.all` las dispara todas juntas.

JavaScript

```
// Esto es lo que hace en 00-seed-all.js (en serie, porque hay
dependencia):
const categoriaIds = await seedCategorias(db);
await seedUsuarios(db);
```

- En `queries.js`, los 4 pipelines **no dependen entre sí** y podrían correr en paralelo.

JavaScript

```
// Versión optimizada con Promise.all
const [stock, historial, ranking, bajoStock] = await Promise.all([
  stockValorizadoPorCategoria(db),
  historialMovimientosProducto(db, 'TEC-001'),
  rankingProductosMasVendidos(db, 5),
  productosBajoStock(db, 10),
]);
```

4. Desestructuración — extraer datos de objetos/arrays

- Permite extraer valores de arreglos u objetos de forma concisa.

JavaScript

```
// connection.js
const { MongoClient } = require('mongodb'); // desestructuración
de un módulo
// data-movimientos.js
const [ana, bruno, carla] = usuarios; // desestructuración
de array
// data-productos.js
const [insumos, tecnologia, limpieza, mobiliario] = categoriaIds;
```

5. Spread operator (...) — copiar/combinar objetos

- Es clave para no mutar documentos de Mongo directamente (inmutabilidad).

JavaScript

```
// Ejemplo de uso para calcular un campo sin tocar el original:
const productoConValor = { ...producto, valorTotal: producto.precio
* producto.stock };
```

6. Métodos de arrays para transformar resultados de Mongo

- `toArray()` devuelve un array común de JS donde puedes usar métodos funcionales.

JavaScript

```
const porSku = (sku) => productos.find((p) => p.sku === sku);
return Object.values(resultado.insertedIds);
```

- Otros métodos comunes sobre resultados de `aggregate`:

JavaScript

```
const nombres = productos.map((p) => p.nombre);
// solo nombres
const activos = productos.filter((p) => p.activo);
// solo activos
const total = productos.reduce((acc, p) => acc + p.stock, 0);
// sumar stock
```

7. Módulos de Node (require, module.exports)

JavaScript

```
// Importar
const { getDb, closeConnection } = require('../db/connection');
// Exportar (patrón que se usa en TODOS los seeds)
module.exports = { seedCategorias, categorias };
```


- `require.main === module` detecta si el archivo se ejecutó directamente o fue importado.

8. Métodos del driver oficial de MongoDB

Método	Para qué	Dónde lo usaste
<code>collection.insertMany(arr)</code>	Insertar varios documentos	en cada <code>data-*.js</code>
<code>collection.deleteMany({})</code>	Vaciar una colección	en cada <code>data-*.js</code> , antes de insertar
<code>collection.find().toArray()</code>	Traer documentos como array	en los seeds, para leer ids existentes
<code>collection.aggregate(pipeline).toArray()</code>	Ejecutar un pipeline	en <code>queries.js</code>
<code>collection.countDocuments()</code>	Contar documentos	en <code>00-seed-all.js</code> , resumen final

Uno que no uso pero es muy común: `findOne()` (trae un solo documento, sin pipeline):

JavaScript

```
const producto = await db.collection('productos').findOne({ sku: 'TEC-001' });
```

PARTE 6

Uso de IA en el desarrollo del proyecto

La herramienta de IA utilizada fue Claude (Anthropic). Se usó en varios momentos del proyecto. El uso de IA en este trabajo fue utilizada de manera que podamos adelantar el trabajo ya que el equipo se encuentra bastante ocupado con otras obligaciones, el uso de IA es solo para agilizar y ayudarnos a trasladar lo aprendido en clases, también lo utilizamos para poder conectar con la base de datos del proyecto. Sin embargo al usar la IA no solo fue mostrar lo que queríamos sino darle el material visto en clases y desde allí poder hacer el proyecto.

También estuvimos utilizando Google Gemini, integrada en docs de google para darle un diseño profesional al PDF.

¿Cómo evaluamos si lo generado era correcto?

Cada fragmento de código fue probado corriendo en nuestra máquina local con MongoDB real. Si el resultado no coincidía con lo esperado (por ejemplo, un pipeline que no devolvía los documentos correctos), lo corregimos e iteramos. No aceptamos código que no pudiéramos ejecutar y verificar nosotros mismos.

¿Qué aprendimos del proceso?

Que la IA es útil para acelerar la escritura de código cuando ya sabés qué querés hacer, pero no reemplaza entender el modelo.

PARTE 7

Reflexión personal

Nos llevamos todo lo aprendido en la cursada, la cual nos dio una base sólida de mongoDB. No cambiaríamos nada de la cursada ya que nos pareció que la metodología de enseñanza fue muy buena y el acompañamiento y dedicación del docente fue muy notoria, estuvo muy presente, para que podamos integrar todos los conceptos.